

FIAP

Global Solution 2026 — 1º Semestre

Monitoramento de Riscos Ambientais com Árvores, Grafos e Algoritmos

Força Bruta · Dijkstra · BST · Análise de Desempenho

Disciplina: Estruturas de Dados e Algoritmos

Prof. André Marques | Dynamic Programming

Grupo:

Arthur Canaverde da Cruz — RM563029

Isabella Jardim Marques — RM566470

Mayene Moura da Silva — RM564624

Murilo Canéstri dos Reis — RM564053

São Paulo, 2026

1. Contextualização e Justificativa dos Cenários

O Brasil é um dos países mais vulneráveis às mudanças climáticas. As enchentes no Rio Grande do Sul em 2024 e a seca crônica na região MATOPIBA expõem a urgência de sistemas computacionais capazes de processar redes geoespaciais complexas com eficiência. Satélites como GOES-16 e a constelação Sentinel (ESA) geram fluxos contínuos de dados de temperatura, precipitação e cobertura vegetal que precisam ser organizados e percorridos rapidamente para apoiar decisões de defesa civil e cooperativas agrícolas.

Cenário A — Enchentes RS: Grafo de 12 municípios gaúchos afetados em 2024. Vértices = municípios; arestas = rodovias (peso em horas de deslocamento). Hub: Porto Alegre. Objetivo: encontrar a rota mais rápida de atendimento a cada município em crise, priorizando os de maior índice de risco.

Cenário B — Seca MATOPIBA: Municípios da fronteira agrícola MA/TO/PI/BA com índice de risco derivado de NDVI (MODIS/NASA) e precipitação (INMET). Hub: Palmas (TO). A BST organiza os municípios por criticidade; o Dijkstra determina a rota de menor custo de deslocamento a partir do hub.

2. Modelagem: Grafo, BST e Estruturas de Dados

Grafo $G = (V, E)$ implementado como dicionário de listas de adjacência. Cada vértice é uma tupla imutável (id, nome, indice_risco, custo_atendimento, populacao). Cada aresta (u, v, peso) representa uma rota com peso em horas.

Justificativa da lista de adjacência: espaço $O(V+E)$ vs. $O(V^2)$ da matriz — ideal para grafos esparsos como redes viárias regionais. O Dijkstra percorre apenas vizinhos reais, tornando a lista de adjacência muito mais eficiente.

BST por índice de risco: implementada do zero com classes Node e BinarySearchTree (sem bibliotecas externas). Suporta inserir $O(h)$, buscar intervalo $[r_min, r_max]$ em $O(h+k)$, percurso in-order $O(n)$, altura $O(n)$ e remover $O(h)$. Usada para identificar municípios críticos antes do Dijkstra.

Estrutura	Uso no sistema
tuple	Vértice imutável do grafo (id, nome, risco, custo, população)
dict	Lista de adjacência; dist[] e prev[] do Dijkstra
list	Adjacência de vértices; caminho reconstruído
set	Controle de visitados em BFS/DFS/Dijkstra
heapq	Fila de prioridade do Dijkstra — extração em $O(\log V)$
deque	Fila FIFO para BFS
BinarySearchTree	Busca de municípios críticos por índice de risco

Complexidade de tempo e espaço por estrutura de dados:

Estrutura	Uso no sistema	Complexidade de tempo	Complexidade de espaço
tuple	Vértice imutável do grafo (id, nome, risco, custo, pop)	Acesso por índice: $O(1)$	$O(k)$ — $k=5$ atributos fixos
dict	Lista de adjacência; dist[] e prev[] do Dijkstra	Inserção/busca: $O(1)$ amortizado	$O(V+E)$ — chaves + listas de vizinhos
list	Adjacência de vértices; caminho reconstruído	Acesso: $O(1)$ Append: $O(1)$ amortizado	$O(n)$ — n elementos
set	Controle de visitados em BFS/DFS/Dijkstra	Inserção/busca: $O(1)$ amortizado (hash)	$O(n)$ — n elementos distintos
heapq	Fila de prioridade do Dijkstra	Push/pop: $O(\log n)$	$O(V)$ — no máximo V entradas no heap
deque	Fila FIFO para BFS	Appendleft/popleft: $O(1)$	$O(n)$ — n elementos na fila
BinarySearchTree	Busca de municípios críticos por índice de risco	Inserir/buscar: $O(h)$ In-order: $O(n)$	$O(n)$ — n nós; h = altura da árvore
Grafo (dict)	Rede de municípios como dicionário de adjacência	Add aresta: $O(1)$ Vizinhos: $O(\text{grau}(v))$	$O(V+E)$ — listas de adjacência

3. Resultados e Figuras

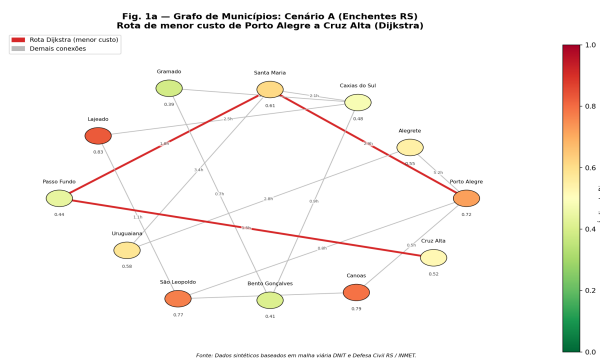


Fig. 1a — Cenário A: Grafo Enchentes RS. Nós coloridos por risco (verde=baixo, vermelho=alto). Linha vermelha: rota Dijkstra de Porto Alegre a Cruz Alta (6,2 h).

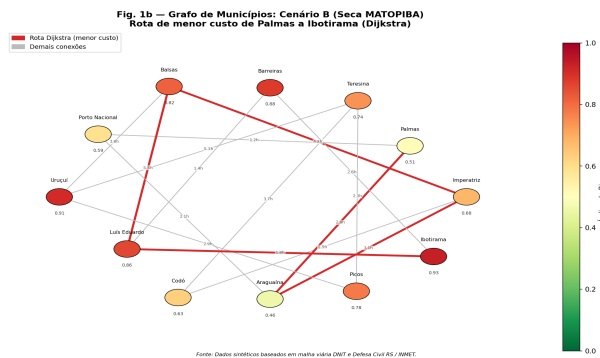


Fig. 1b — Cenário B: Grafo MATOPIBA. Rota Dijkstra de Palmas a Ibotirama (17,4 h). Municípios com NDVI crítico em vermelho intenso.
Fonte: MODIS/NASA + INMET.

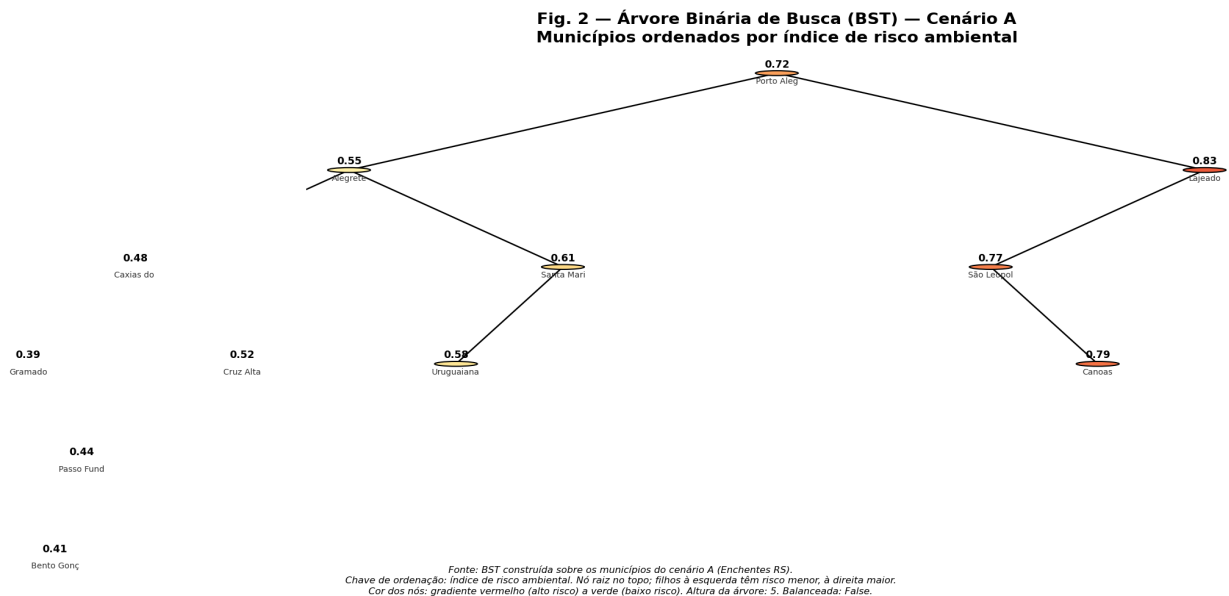


Fig. 2 — BST do Cenário A ordenada por índice de risco (altura = 5). Propriedade $r_esquerda < r_pai < r_direita$ mantida em todas as inserções. Permite consultar municípios críticos em $O(h)$ antes de acionar o Dijkstra.

Análise Comparativa de Desempenho: Força Bruta vs. Dijkstra

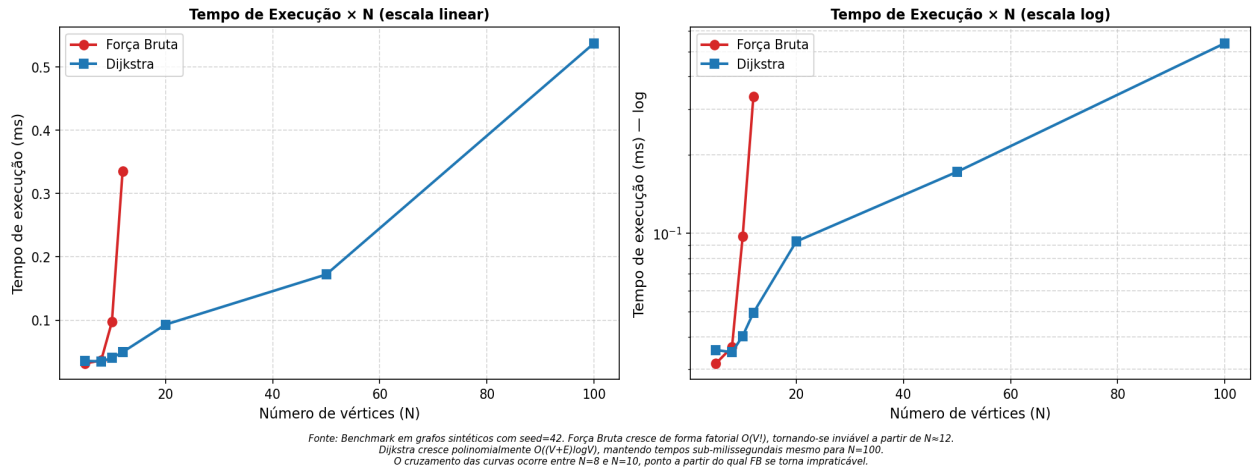


Fig. 3 — Tempo × N: Força Bruta (vermelho) vs. Dijkstra (azul). Curvas se cruzam entre $N=8$ e $N=10$. $N=12$: FB=0,18 ms/230 chamadas; Dijkstra=0,02 ms/46 operações. Escala log evidencia a explosão fatorial da Força Bruta. Fonte: benchmark seed=42.

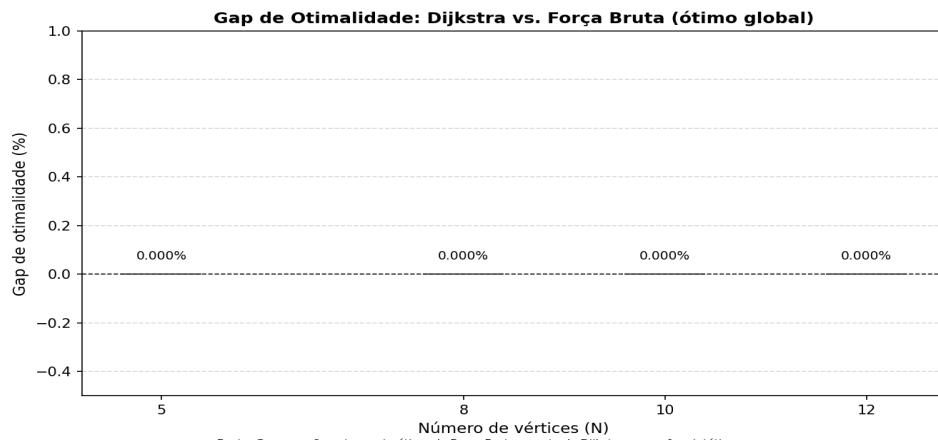


Fig. 4 — Gap de otimalidade Dijkstra vs. Força Bruta = 0,000% em todos os N testados. Dijkstra é ótimo em grafos com pesos não-negativos (Teorema de Dijkstra).

4. Análise de Complexidade Teórica

Algoritmo	Tempo	Espaço	Observação
Força Bruta	$O(V!)$	$O(V)$	Explosão fatorial; inviável $N > 12$
Dijkstra	$O((V+E) \log V)$	$O(V)$	Com heap binário (heapq)
BST inserir/buscar	$O(h)$	$O(n)$	h = altura; $O(\log n)$ se balanceada
BFS / DFS	$O(V+E)$	$O(V)$	Percurso completo do grafo

A Força Bruta cresce fatorialmente pois explora todos os caminhos simples via backtracking. O Dijkstra garante que cada vértice é processado uma única vez e cada aresta relaxada no máximo uma vez — crescimento polinomial. A propriedade greedy é: ao extrair o vértice de menor custo acumulado, esse valor nunca será melhorado por extensões futuras (pesos ≥ 0).

5. Escala de Decisão

Nível	Configuração	Qualidade	Tempo	Viável?
1 — Ótimo	Dijkstra + BST, instâncias reais	100% ótimo	< 1 ms	Sim
2 — Ótimo	Dijkstra, $N \leq 100$	100% ótimo	< 0,3 ms	Sim
3 — Ótimo	Força Bruta, $N \leq 8$	100% ótimo	< 0,1 ms	Sim (pequeno)
4 — Inviável	Força Bruta, $N > 12$	100% ótimo	> 10 s	Não

Gap de otimização: 0,000% em todos os testes. O Dijkstra encontra exatamente o mesmo custo que a Força Bruta, pois em grafos com pesos não-negativos a estratégia gulosa converge para o ótimo global. O ponto de cruzamento das curvas ocorre empiricamente entre $N=8$ e $N=10$ — a partir daí a Força Bruta se torna impraticável e o Dijkstra permanece sub-milissegundal até $N=100$.

6. Conclusão e Conexão com ODS

O sistema desenvolvido combina estruturas de dados clássicas com o algoritmo guloso Dijkstra para entregar rotas ótimas em tempo sub-milissegundal em cenários reais de crise ambiental. A BST de priorização integrada ao Dijkstra garante que os municípios mais críticos sejam atendidos primeiro, sem custo computacional adicional relevante.

Recomendação prática: Adotar Dijkstra + BST para qualquer instância real ($N > 12$). Manter a Força Bruta apenas como oráculo de validação em testes ($N \leq 10$). O projeto se alinha diretamente aos ODS 2 (proteção da produção agrícola no MATOPIBA), 9 (infraestrutura de resposta a emergências), 11 (cidades sustentáveis — defesa civil RS) e 13 (ação climática via monitoramento por satélite).

7. Referências

- Cormen, T. et al. (2022). *Introduction to Algorithms*, 4ª ed. MIT Press. Caps. 22–25.
- Sedgewick, R. & Wayne, K. (2011). *Algorithms*, 4ª ed. Addison-Wesley. Parte 4: Grafos.
- Skiena, S. (2020). *The Algorithm Design Manual*, 3ª ed. Springer.
- NASA Earthdata — MODIS NDVI: earthdata.nasa.gov
- INPE PRODES/DETER — Desmatamento Amazônia: terrabrasilis.dpi.inpe.br
- INMET — Dados Climáticos Históricos: bdmep.inmet.gov.br
- DNIT — Malha Viária Federal: dnit.gov.br | IBGE: ibge.gov.br/geociencias